

After Acceptance: A Claim Graph for Residual Languages and Weird-Machine Claims, Calibrated on eBPF

Chengao Zhang
Independent Researcher
emtanling@gmail.com

Abstract—Language-theoretic security models an input boundary as a recognizer for a formal language and downstream processing as interpretation. At a program-verifier boundary, the verifier is an outer recognizer of program artifacts; an accepted artifact may in turn interpret data and drive runtime-operation words, while any computed report is a separate, explicitly declared abstraction. We introduce a five-node claim graph separating artifact acceptance (A), a same-suffix causal state distinction (C), bounded programmability (P), output-witnessed computed-report non-factorization (R), and a policy/threat obligation (W). A future-observation quotient and behavioral factorization criterion distinguish the accepted-artifact-indexed causal language from the report-relative residual language. The criterion is report-instance-relative, applies only under an explicit unique-cell report map on the chosen context fiber, and does not assert that every safety-sound incomplete verifier hosts a weird machine.

We give proof obligations for a uniformly controlled, observable, resettable gate and one fixed accepted interpreter over a bounded circuit-description domain. Prefix induction yields bounded composition under exact scheduling, admissibility, serialization, and frame-preservation premises. In a Linux eBPF calibration, under the stated map-update, reset, and no-interference contract, a dedicated preallocated non-LRU two-entry hash map serves as a NAND gate: reset leaves one sentinel entry; input bits select existing- or fresh-key updates; and the second update’s success predicate supplies the output. One fixed accepted program consumes canonical NAND DAGs with at most 64 inputs, 512 gates, and 578 live wires.

A source-snapshotted Linux/aarch64 run covers named and random circuits, joint bounds, serial reuse, mechanism controls, and malformed descriptors. A separate author-run semantic auditor reconstructs descriptors and circuit semantics, and a self-issued manifest checks bundle integrity. This interpreter carrier records A, gives a conditional C witness under the declared concrete-service/no-interference contract, and supports P under additional implementation premises; it establishes neither R nor W. A fixed auxiliary executable instance establishes $R(M_{linux_r_aux_v1})$ for its own computed report. Separately, the frozen stock-Linux V1 experiment on a different accepted XDP object captures a successful exact-level-0 `states_equal` check followed by an `is_state_visited` prune and two different same-suffix samples, but the evidence-bounded reassessment does *not* promote its historical adapter result to the real system: V1 outcome eligibility remains `NOT_ESTABLISHED`, so the exact V1 operational-prune query is `UNKNOWN`. A newer prospective Stock-R V2 experiment closes that missing premise only for its own exact query: its runner seals object, translated-bytecode, BTF, kernel, source-closure, checker, and runtime identities, writes `proof/must-outcome-proof.json` and `proof/history-case-binding.json`, and checks that the selected prune histories are the proof’s

two cases at one frontier, report cell, and suffix. A fresh run on stock Ubuntu kernel 6.17.0-35-generic reports `outcome_eligibility.status = ESTABLISHED`, `method = MUST_OUTCOME_PROOF_WITH_HISTORY_CASE_BINDING`, `assessment.status = NONFACTORING`, and `assessment.scope = EXACT_STOCK_R_V2_QUERY`. A generic evidence-graph/proof-DAG checker then reproduces V1 as `BLOCKED/INCONCLUSIVE` and V2 as exact `CERTIFIED/NONFACTORING`; twelve hostile mutations block five unsupported requested lifts and reject seven proof-wide or integrity/dependency attacks. A guarded Contextual Residual Lifting (CRL) extension adds a verifier-contract residuality theorem and a checked `DERIVED_CONTEXTUAL` transport chain. On two generated VM targets whose translated-bytecode digests differ from the V2 source and from each other, the checker emits separate exact `AT(target)/TRANSPORTED` certificates and contextual hostile matrices reject scope, integrity, selection, derivation-chain, and circularity attacks. The carriers are not combined: the auxiliary R result, Stock-R V2, and the contextual targets are not linked to the interpreter’s P evidence, and no result establishes W or a weird machine.

Index Terms—language-theoretic security, recognizers, residual languages, weird machines, certifying methodology, program verification, abstract interpretation, eBPF

I. INTRODUCTION

Language-theoretic security treats an input boundary as a language-recognition boundary: downstream processing is interpretation, and the input supplies the interpreted program [1]–[4]. We use the same recognizer-shaped model for program verification. Our question begins after acceptance. A verifier accepts a program artifact, but the accepted program can still drive a language of stateful helper and service operations. Which claims are needed before that downstream language can be called residual, programmable, shape-induced, or a weird machine? In this layered view, acceptance, interpretation, and report abstraction answer different questions: membership in L_V , behavior in I , and grouping by Report_V , respectively.

The artifact language and the post-acceptance operation language have different carriers. A verifier can recognize bounded execution, typed helper use, and memory-access discipline without certifying the complete relation implemented through every documented runtime service. The accepted program may select operations, retain service state, observe returns, reset state, and compose effects. None of this alone implies a verifier

defect. It first establishes a downstream operation language indexed by an accepted artifact.

The paper organizes the evidentiary burden as a claim graph:

Each row in Table I has its own artifact, execution carrier, report interface, and evidence type. The `wm_circuit` row carries the paper’s A/C/P construction. The auxiliary row supplies a positive R result only for a custom report-producing recognizer. The V1 M_{Linux} row supplies a frozen observation bundle; the distinct legacy construction $M_K^{\text{legacy}} = M_{\text{adapter}}$ has a factorization result, not an R certificate for M_{Linux} . The V2 row supplies a proof-bound exact-query operational-prune result for a different accepted object and witness. The contextual row supplies a target-bound transport certificate derived from V2, not a new source proof or a family theorem. No column may be read by combining cells across rows, and no evaluated row establishes W.

Nodes C and R must not share one name. We call the accepted-artifact-indexed, same-suffix language L_{causal} . A word enters the report-relative residual language L_{res}^R only with an actual computed-cell collision under the admissibility conditions below. P and R branch after C. A policy-level weird machine requires P, W, linkage to the same encoded computation, and unintendedness; a *contract-shape-induced recognizer-relative* classification additionally requires R, documented semantics, report conformance, and granularity evidence. The graph turns “weird machine” from a rhetorical label into explicit, auditable obligations that support both positive certificates and principled non-classification.

Programmability is a separate constructive problem. A state distinction may be dead, uncontrolled, one-shot, or impossible to compose. We therefore require a program-visible readout, one uniform input dispatcher, a reset to a canonical class, and one fixed accepted interpreter with exact scheduling and frame preservation over an independently declared bounded descriptor domain. The resulting composition theorem is a proof-obligation decomposition, not a universal necessity theorem.

Our calibration case is Linux eBPF. A fixed artifact uses a dedicated two-entry hash map as a saturating resource. After reset, one sentinel is live. A zero bit updates the sentinel; a one bit inserts a fresh input-specific key. The second input-conditioned update succeeds except on input (1, 1), yielding NAND. Ordinary bytecode still validates descriptors, selects keys, routes wires, controls the loop, and stores outputs. A host parser normalizes textual WMC1 into maps, while the verifier accepts only the fixed eBPF artifact.

The interpreter case is deliberately diagnostic. It records A, gives a conditional C witness under the declared service contract, and supports P under stated implementation premises, even though ordinary bytecode easily expresses the same function. It also shows why P cannot be promoted to R or combined with an absent W obligation. Its retained verifier log is not a computed-cell extractor, and the offline run supplies no violated policy. The separate M_{Linux} capture supplies evidence of an actual prune event for `rac_single`, not for the interpreter.

The paper contributes:

- 1) **A recognizer–interpreter–report claim graph and typed languages.** We distinguish accepted artifacts, causal runtime words, report-relative residual words, bounded programmability, and policy/threat obligations. A future-observation quotient and factorization criterion make node R testable, while countermodels show that the branches do not collapse.
- 2) **A bounded composition argument.** E1–E3 and the precise E4–D interpreter obligations isolate local gate correctness, reset, scheduling, and global frame conditions. Prefix induction proves the functional result; safety is a separate optional premise.
- 3) **An eBPF calibration case.** A saturating-rank NAND basis and fixed interpreter target $D_{64,512}$. The recorded suite covers named and fixed-seed random DAGs, deep and joint boundaries, a zero-gate case, serial reuse, mechanism controls, and malformed descriptors.
- 4) **Carrier-separated evidence and exact certifying controls.** A fixed auxiliary recognizer/service tuple emits a report and checks a finite report-cell collision with four negative controls. A separate stock-Linux V1 capture records a real exact-level-0 prune and two common-suffix samples; its legacy adapter has a finite-model factorization failure, but the evidence-bounded reassessment in Section 5.7 classifies the V1 real-system query as UNKNOWN rather than R. Stock-R V2 adds a prospective runner, runtime replication, a checked must-outcome proof, and a checked history-case binding, establishing NONFACTORING only for EXACT_STOCK_R_V2_QUERY. A generic evidence-graph/proof-DAG checker preserves this V1-blocked/V2-certified distinction and fails closed under twelve hostile mutations. CRL then derives two separate DERIVED_CONTEXTUAL target certificates from the exact V2 source certificate without consuming a target terminal verdict. None of these stock-Linux rows is linked to the interpreter’s P evidence.

The empirical scope is one privileged, offline Linux/aarch64 kernel build family, four claim-bearing accepted eBPF artifacts across the interpreter, V1, V2, and contextual-target stock-Linux carriers, three additional accepted interpreter-control variants, and one accepted auxiliary custom-recognizer artifact. The result is not a general Linux report-opacity theorem, concurrent deployment result, artifact-parametric compiler, unbounded machine, vulnerability, or policy-level weird machine.

II. RECOGNIZER-RELATIVE RESIDUAL LANGUAGES

A. Recognition, execution, and report

Let V be a recognizer over program artifacts and I the concrete execution semantics over state carrier Σ_I and trace carrier $\mathcal{T}_I$, including the instruction engine, helpers, stateful services, and relevant environment. Thus $\text{Tr}_I(P) \subseteq \mathcal{T}_I$. The accepted artifact language is

$$L_V = \{P \mid V(P) = \text{accept}\}. \quad (1)$$

TABLE I
CARRIER-BY-NODE EVIDENCE STATUS.

Carrier	A/C/P/R/W status	Evidentiary interpretation
wm_circuit	A recorded; C conditional under the declared service contract; P supported under source/object, serialization, and frame premises; R and W not established	construction plus audited regression evidence; no computed-cell extractor at the interpreter frontier
$M_{linux_r_aux_v1}$	A and C established within the finite custom recognizer; R established for its computed custom report; P and W not established	executable finite-model certificate; no refinement or bisimulation claim to stock Linux
M_{Linux}^{V1} (rac_single; V1)	A recorded for the frozen tuple; two sampled outcomes under the declared history relation; P and W not established; R UNKNOWN under the exact evidence-bounded query	real prune capture plus the distinct M_K^{legacy} adapter result; no must-outcome proof and no Linux functional-report contract
M_{Linux}^{V2} (rac_v2; prospective Stock-R V2)	A recorded for the controlled tuple; operational prune, runtime replication, checked must-outcome proof, and checked history-case binding establish exact-query NONFACTORING; P and W not established	proof-and-binding result for the declared operational-prune report on the V2 array-map witness; no Linux functional-report contract and no broader-runs claim
$M_{Linux}^{V2.ctx}$ (rac_v2_ctx)	A recorded for two generated contextual targets; CRL derives exact AT(target)/TRANSPORTED NONFACTORING from the exact V2 source certificate; P and W not established	DERIVED_CONTEXTUAL certificate chains for two targets whose translated-bytecode digests differ from V2 and from each other; no family, compiler-correctness, or general Linux claim

For an optional set $\text{Safe} \subseteq \mathcal{T}_I$ of permitted concrete traces, the boundary is safety-sound when

$$\forall P \in L_V. \text{Tr}_I(P) \subseteq \text{Safe}. \quad (2)$$

Safety soundness is a premise, not an inference from one successful load. It is also distinct from completeness of an abstract transformer. Abstract interpretation relates concrete sets and abstract elements through abstraction and concretization maps [5], but a verifier's accepted language, its computed report, transfer soundness, and completeness are different judgments. In particular, the best abstraction of a singleton need not be the analyzer-computed cell at a joined control frontier.

When a report is in scope, let $\text{Report}_V(P, \ell) \subseteq A_\ell$ be the finite set of abstract cells actually computed at frontier ℓ , with a declared concretization $\gamma_\ell : A_\ell \rightarrow \mathcal{P}(\Sigma_I)$. Frontier coverage requires

$$\text{Reach}_I(P, \ell) \subseteq \bigcup_{a^\# \in \text{Report}_V(P, \ell)} \gamma_\ell(a^\#). \quad (3)$$

Two concrete states are jointly covered only when one computed cell contains both. This point rules out a common but invalid shortcut: showing that two values have similar printed log text does not identify a computed abstract cell, a concretization, or joint coverage.

B. Causal words

For $P \in L_V$ and frontier ℓ , let $\Sigma_{\text{op}}(P)$ be a finite alphabet of program-controllable runtime operations, and let $W_{\text{run}}^\ell(P) \subseteq \Sigma_{\text{op}}(P)^*$ be the words accepted code can execute from ℓ . Calling every such word residual would rename ordinary trace semantics, so node C uses a same-suffix causal test without yet asserting report omission.

Fix before comparison an observation contract

$$K_{\text{obs}} = (\rho_{\text{obs}}, \text{Obs}, \text{Slice}, \text{Env}). \quad (4)$$

Here $\rho_{\text{obs}} : \Sigma_I \rightarrow R_{\text{obs}}$ selects the candidate state, $\text{Obs} : \mathcal{T}_I \rightarrow O_{\text{obs}}$ projects a complete concrete trace, Slice assigns to each word w a typed context projection $\text{ctx}_w : \Sigma_I \rightarrow C_w$, and Env is the declared set of fixed environment instances. The context conservatively contains every non-selected component read by the suffix or observer; an environment instance fixes the relevant resource configuration, schedule, nondeterminism, and external-interference choice.

Write $\llbracket w \rrbracket_{I,e}(\sigma) = (\tau, \sigma')$ for a defined, terminating suffix execution in environment e . The contract is *sound for w on $X \subseteq \Sigma_I$* when, for every $e \in \text{Env}$ and $\sigma, \sigma' \in X$ satisfying

$$\rho_{\text{obs}}(\sigma) = \rho_{\text{obs}}(\sigma') \quad \text{and} \quad \text{ctx}_w(\sigma) = \text{ctx}_w(\sigma'), \quad (5)$$

the two suffix executions have the same definedness, and, whenever $\llbracket w \rrbracket_{I,e}(\sigma) = (\tau, \sigma_1)$ and $\llbracket w \rrbracket_{I,e}(\sigma') = (\tau', \sigma'_1)$, then $\text{Obs}(\tau) = \text{Obs}(\tau')$. Thus $(\rho_{\text{obs}}, \text{ctx}_w, e)$ contains every factor that may affect the declared observation. Without this noninterference condition no C witness is admitted.

Definition 1 (causal state-mediated word family). A word w is causal at (P, ℓ) when $P \in L_V$, $w \in W_{\text{run}}^\ell(P)$, K_{obs} is sound for w on $\text{Reach}_I(P, \ell)$, and there are $e \in \text{Env}$ and $\sigma_0, \sigma_1 \in \text{Reach}_I(P, \ell)$ for which $\llbracket w \rrbracket_{I,e}(\sigma_i) = (\tau_i, \sigma'_i)$ are both defined and terminating, and

$$\begin{aligned} \text{ctx}_w(\sigma_0) &= \text{ctx}_w(\sigma_1), \\ \rho_{\text{obs}}(\sigma_0) &\neq \rho_{\text{obs}}(\sigma_1), \\ \text{Obs}(\tau_0) &\neq \text{Obs}(\tau_1). \end{aligned} \quad (6)$$

Define the dependent tagged family

$$L_{\text{causal}}(V, I; K_{\text{obs}}) = \{(P, \ell, w) \mid w \text{ is causal at } (P, \ell)\}. \quad (7)$$

The artifact and frontier tags are part of the type. With fixed, decodable prefix encodings for artifacts, frontiers, and operation symbols, the family can be embedded in an ordinary language over one finite alphabet; the dependent notation

keeps the carriers visible. A host descriptor is neither another element of L_V nor automatically a runtime word. It first becomes configuration, the accepted interpreter induces a schedule, and only a same-suffix witness enters L_{causal} .

The definition is observer- and slice-relative. Deleting an earlier input from the context after seeing the result would make the test circular. The `wm_circuit` interpreter case therefore declares its suffix from source semantics before comparison; its translated dump is retained for manual inspection rather than consumed by an automated slice checker. The separate `rac_single` carrier instead uses the hash-bound path-correlation check described in Section 5.7.

C. Behavioral quotient and report-relative residual language

Fix a deterministic discipline

$$D = (X_D, S_D, A_D, O_D, \delta_D, \lambda_D, s_D, \text{Obs}_D) \quad (8)$$

with an admissible concrete region $X_D \subseteq \Sigma_I$, state carrier S_D , operation alphabet A_D , output alphabet O_D , observer $\text{Obs}_D : O_D^* \rightarrow O_{\text{obs}}$, and same-domain partial functions

$$\delta_D : S_D \times A_D \rightarrow S_D, \quad \lambda_D : S_D \times A_D \rightarrow O_D. \quad (9)$$

Whenever D is used at (P, ℓ) , fix an injective operation encoding $\iota_{P, \ell} : A_D \rightarrow \Sigma_{\text{op}}(P)$ and extend it homomorphically. The pair $(s_D, \iota_{P, \ell})$ is *operationally adequate on X_D* when, for every $\sigma \in X_D$ and $a \in A_D$, concrete execution of $\iota_{P, \ell}(a)$ exists exactly when $\delta_D(s_D(\sigma), a)$ is defined; whenever $\sigma \xrightarrow{\iota_{P, \ell}(a)/o} \sigma'$, where $o \in O_D$ is the complete declared observation of that encoded operation, then $\sigma' \in X_D$ and

$$\lambda_D(s_D(\sigma), a) = o, \quad s_D(\sigma') = \delta_D(s_D(\sigma), a). \quad (10)$$

Thus, within X_D , the encoding is a semantic renaming rather than an arbitrary map: one projected state cannot hide different definedness, outputs, or successor projections. Every environment choice that affects a transition is fixed by D or included in S_D .

Let $\text{Out}_D(r, w)$ be the partial complete output word obtained by iterating (δ_D, λ_D) , write $\text{Def}_D(r, w)$ for $\text{Out}_D(r, w) \downarrow$, and take the continuation universe to be $\mathcal{W}_D = A_D^*$; infeasible words are represented by undefined output. For $r, r' \in S_D$, define

$$\begin{aligned} r \sim_D r' &\iff \forall w \in \mathcal{W}_D. \\ &\left[\text{Def}_D(r, w) \iff \text{Def}_D(r', w) \right] \\ &\wedge \left[\text{Def}_D(r, w) \Rightarrow \text{Out}_D(r, w) = \text{Out}_D(r', w) \right]. \end{aligned} \quad (11)$$

Because $\mathcal{W}_D = A_D^*$ is closed under left-prefixing by an operation, $\sim_D$ is a right congruence: defined matching a -steps lead to equivalent successor states, since every successor continuation w is tested as aw . This follows the classical continuation-equivalence lineage of Nerode and Mealy machines [6], [7]; the partial-output setting and observer contract here are paper-specific. If $Q_D = S_D / \sim_D$ is finite, execution restricted to X_D induces a partial Mealy transducer over quotient states. Observational completeness and generalized strong

preservation likewise ask whether an abstraction preserves a selected observation or specification language [8], [9]. Our test is narrower: it concerns cells actually computed for one accepted artifact at one frontier. It is a semantic quotient test, not a claim to introduce a new general completeness theory.

For a concrete set F , define the common enabled continuations

$$\mathcal{W}_D(F) = \{w \in \mathcal{W}_D \mid \forall \sigma \in F. \text{Out}_D(s_D(\sigma), w) \downarrow\}. \quad (12)$$

Proposition 1 (behavioral factorization on a context fiber).

Fix accepted P , frontier ℓ , discipline D , and observation contract K_{obs} . Let $\emptyset \neq F \subseteq \text{Reach}_I(P, \ell) \cap X_D$. Fix an injective operation encoding $\iota_{P, \ell} : A_D \rightarrow \Sigma_{\text{op}}(P)$ and its homomorphic extension to words. Assume

$$\iota_{P, \ell}(\mathcal{W}_D(F)) \subseteq W_{\text{run}}^\ell(P), \quad (13)$$

that K_{obs} is sound on F for every encoded word $\iota_{P, \ell}(w)$ with $w \in \mathcal{W}_D(F)$, and that

$$\forall w \in \mathcal{W}_D(F). \forall \sigma, \sigma' \in F. \text{ctx}_{\iota_{P, \ell}(w)}(\sigma) = \text{ctx}_{\iota_{P, \ell}(w)}(\sigma'). \quad (14)$$

Require observation compatibility: whenever encoded $w \in \mathcal{W}_D(F)$ executes from $\sigma \in F$ with concrete trace τ , the following right-hand side is defined and

$$\text{Obs}(\tau) = \text{Obs}_D(\text{Out}_D(s_D(\sigma), w)). \quad (15)$$

Define

$$\beta_D(\sigma) = [s_D(\sigma)]_D, \quad F_{a^\#} = F \cap \gamma_\ell(a^\#). \quad (16)$$

Finally, require the unique-cell condition

$$\forall \sigma \in F. \exists! a^\# \in \text{Report}_V(P, \ell). \sigma \in \gamma_\ell(a^\#), \quad (17)$$

and let $\pi_R : F \rightarrow \text{Report}_V(P, \ell)$ map each state to that unique computed cell. Then the report factors the future-observation quotient on F ,

$$\exists h : \pi_R(F) \rightarrow Q_D. \beta_D = h \circ \pi_R, \quad (18)$$

if and only if

$$\forall a^\# \in \pi_R(F). |\beta_D(F_{a^\#})| \leq 1. \quad (19)$$

Consequently, report non-factorization on F is equivalent to a unique computed cell containing two states from different $\sim_D$ classes.

Proof. If $\beta_D = h \circ \pi_R$, states with the same unique report label have the same quotient class, giving the cardinality bound. Conversely, the bound makes $h(a^\#)$ the unique element of $\beta_D(F_{a^\#})$ for every $a^\# \in \pi_R(F)$; this is well-defined and yields the factorization. Negating either equivalent condition gives the collision statement. $\square$

Write $\text{Adm}(P, \ell, D, F; K_{\text{obs}})$ when all of the following hold: $P \in L_V$; $(s_D, \iota_{P, \ell})$ is operationally adequate on X_D ; $\emptyset \neq F \subseteq \text{Reach}_I(P, \ell) \cap X_D$; the runtime-word inclusion, observer compatibility, sound observation contract, common-context condition, and unique-cell condition above all hold. The last condition makes the nonempty $F_{a^\#}$ a disjoint cover.

Reports with overlapping labels require a separately declared membership-signature map and are outside this criterion.

Admissibility is an internal semantic typing condition, not a safeguard against post-selection. Every positive instantiation must therefore declare whether D , F , the observer, suffix, and report interface were fixed prospectively or selected retrospectively after the distinguishing execution was known. A prospective or report-general claim additionally requires selection provenance independent of the witnessed output, such as a predeclared protocol or an externally specified report contract. A retrospective tuple can satisfy Definition 2 only when every semantic and evidence premise is independently established; even then, that fact alone cannot characterize the recognizer’s intended report, a broader execution domain, or a population of implementations.

Definition 2 (report-relative residual language). Fix an admissible tuple. A dependent tagged word $(P, \ell, D, F, a^\#, w)$ is *output-witnessed report-relative residual* when:

$$\begin{aligned} (P, \ell, \iota_{P, \ell}(w)) &\in L_{\text{causal}}(V, I; K_{\text{obs}}), & w &\in \mathcal{W}_D(F), \\ \sigma_0, \sigma_1 &\in F \text{ are Definition 1 witnesses for } \iota_{P, \ell}(w), \\ \pi_R(\sigma_0) &= \pi_R(\sigma_1) = a^\#, & \beta_D(\sigma_0) &\neq \beta_D(\sigma_1). \end{aligned} \quad (20)$$

$L_{\text{res}}^R(V, I, \text{Report}; K_{\text{obs}})$ is the union of these tagged words over tuples satisfying Adm . Operational adequacy and observer compatibility make different admitted concrete observations imply different quotient classes; the explicit inequality binds the definition to Proposition 1. The quotient also distinguishes definedness, so report non-factorization can arise without a jointly terminating output-distinguishing word. Accordingly, L_{res}^R is the output-witnessed subset of report-factorization failures, not a converse characterization of all failures. It is a frontier collision, not a claim that a complete whole-program report can never recover the final function.

Instantiating Definition 2 requires an extractor for actual computed cells, a declared concretization, a unique-cell partition on the chosen fiber (or a separately defined alternative report map), and the remaining admissibility premises. The retained `wm_circuit` Linux log does not provide these objects, so that interpreter carrier gives only a conditional L_{causal} witness under its declared service contract and does not establish L_{res}^R . Section 5.7 documents a different, finite `rac_single` legacy construction and explains why its V1 evidence does not establish the real-system premises.

D. Shape, defect, and policy

A defect-induced gap uses behavior that violates the declared recognizer, report, or runtime contract. A documented, safety-preserving witness in L_{res}^R for a relation the report intends to certify is evidence for a contract-shape gap, not by itself a shape proof: report conformance and granularity must rule out a faulty transformer, join, or extractor. A policy-level weird machine additionally needs linked actor control, a policy-excluded effect, and unintended interpretation. An ordinary stateful API can therefore implement a transducer without satisfying either classification.

III. BOUNDED STATE-MEDIATED CIRCUIT REALIZATION

A. From a distinction to a reusable gate

A causal word can expose one distinction without supporting repeated computation. Fix a deterministic gate discipline D , one canonical reset class $q_0 \in Q_D$ identified with its subset of S_D , and a nonempty admissible set $\text{Adm}_G \subseteq X_D$. A gate basis $(P_G, \text{reset}, G, \text{observe}, D, \text{Adm}_G)$ uses one accepted artifact P_G , a dispatcher G selecting a complete word $u_G(x) \in A_D^*$ for each $x \in \{0, 1\}^2$, and a partial readout $\text{observe} : O_D^* \rightarrow \{0, 1\}$. For this basis, $O_{\text{obs}} = \{0, 1\}$ and $\text{Obs}_D = \text{observe}$. Its predeclared operation encoding into $\Sigma_{\text{op}}(P_G)$ is operationally adequate on X_D . The basis satisfies:

E1 — causal basis and observation. There exist inputs x_0, x_1 , prefixes $p_0, p_1 \in A_D^*$, one common remaining suffix $w \in A_D^*$, reset-class states $\eta_0, \eta_1 \in \text{Adm}_G$, and one internal frontier ℓ_G such that

$$u_G(x_i) = p_i w, \quad s_D(\eta_i) \in q_0. \quad (21)$$

Executing each p_i reaches a state σ_i at ℓ_G ; the two σ_i are Definition 1 witnesses for the encoded common suffix w . If its traces are τ_i , then the complete gate readout is precisely that causal observation:

$$\text{observe}(\text{Out}_D(s_D(\eta_i), u_G(x_i))) = \text{Obs}(\tau_i), \quad i \in \{0, 1\}. \quad (22)$$

Accepted code uses this readout as the gate result that E4-D writes; it is not an unrelated probe.

E2 — uniform input control. The dispatcher G in P_G reads runtime bits $x \in \{0, 1\}^2$. For every $\sigma \in \text{Adm}_G$ with $s_D(\sigma) \in q_0$, $\text{Out}_D(s_D(\sigma), u_G(x))$ and its readout are defined and

$$\text{observe}(\text{Out}_D(s_D(\sigma), u_G(x))) = g(x) \quad (23)$$

for one fixed $g : \{0, 1\}^2 \rightarrow \{0, 1\}$. The experimenter is not an external oracle choosing a different constant program for each input.

E3 — reset. The reset word satisfies $\text{reset} \in A_D^*$. From every $\sigma \in \text{Adm}_G$, it is defined, preserves declared wire cells, and terminates in $\tau \in \text{Adm}_G$ with $s_D(\tau) \in q_0$. If two pre-states in Adm_G agree on those wire cells and the fixed context and reset to τ_0, τ_1 , then for every $x \in \{0, 1\}^2$,

$$\text{Out}_D(s_D(\tau_0), u_G(x)) = \text{Out}_D(s_D(\tau_1), u_G(x)). \quad (24)$$

Without E1, a hidden difference has no program-visible effect. Without E2, the experimenter may supply the truth table. Without E3, the channel may be one-shot. A fourth condition below composes the gate without placing circuit semantics in an external oracle.

B. Descriptor domain

The artifact uses the bounded domain $D_{64,512}$. A descriptor is

$$d = (m, n, (s_i^0, s_i^1)_{0 \leq i < n}), \quad (25)$$

with $0 \leq m \leq 64$, $0 \leq n \leq 512$, and

$$0 \leq s_i^b < 2 + m + i \quad (26)$$

for each gate i and operand b . Write $m(d) = m$ and $n(d) = n$. Wire 0 is constant zero, wire 1 is constant one, primary inputs occupy wires 2 through $2 + m - 1$, and gate i writes canonical destination $2 + m + i$. Thus the maximum number of live canonical wires is $2 + 64 + 512 = 578$ and the highest valid wire index is 577.

For $x \in \{0, 1\}^{m(d)}$ and gate function g , $\text{Eval}_g(d, x)$ initializes the constants and input vector, then extends the wire vector in descriptor order:

$$\nu[2 + m + i] = g(\nu[s_i^0], \nu[s_i^1]). \quad (27)$$

Its result is the complete canonical vector $(\nu[0], \dots, \nu[1 + m(d) + n(d)])$, the same type later returned by WireObs_d .

The host encoding $\text{Enc}_U(d, x)$ writes a normalized descriptor, inputs, and control record into maps. Textual WMC1 is a host serialization. Neither WMC1 nor d is a BPF program accepted by V ; $\text{Sched}_U(d, x)$ is the operation schedule induced when accepted P_U reads the encoding.

For $c = \text{Enc}_U(d, x)$, let $\text{PhysRun}_U(c) = (s, k, \mu)$ contain final status, completed-iteration count, and physical map state. Define the descriptor-relative canonical observation

$$\text{WireObs}_d(\mu) = (\mu[0], \dots, \mu[1 + m(d) + n(d)]) \quad (28)$$

and the status-masked interface

$$\text{Run}_{U,d}(c) = \begin{cases} (s, \text{WireObs}_d(\mu)), & s = \text{OK}, \\ (s, \perp), & s \neq \text{OK}. \end{cases} \quad (29)$$

The host may project requested output wires only from an OK result. This semantic rule does not assert that stale physical cells are erased.

E4-D — bounded data-parametric interpretation. One fixed artifact P_U with a gate basis whose artifact component is the same P_U discharges E4-D for $D_{64,512}$ when:

- 1) for the fixed map definitions and recorded load environment, $P_U \in L_V$ independently of descriptor contents d and x ;
- 2) every valid $c = \text{Enc}_U(d, x)$ establishes a pre-callback state $\mu^{(0)}$ with $\mu^{(0)}[0] = 0$, $\mu^{(0)}[1] = 1$, and $\mu^{(0)}[2 + j] = x_j$ for $0 \leq j < m(d)$, executes exactly callback positions $0, \dots, n(d) - 1$ in order, and terminates with $\text{PhysRun}_U(c) = (\text{OK}, n(d), \mu)$;
- 3) one external critical section covers setup, invocation, and readback for every shared map in the footprint;
- 4) every iteration i validates canonical form and copies exactly its two earlier source wires; the complete concrete state σ_i immediately before reset belongs to Adm_G ; the iteration then resets and invokes the E1–E3 gate, writes the resulting g bit only to destination $2 + m(d) + i$ and declared audit cells, and preserves the descriptor and all earlier wires; and
- 5) no execution performs more than 512 iterations, while malformed core-ABI controls or descriptor configurations covered by the declared validator terminate with non-OK status, at most 512 completed iterations, and semantic result $(s, \perp)$.

These clauses are deliberately proof obligations. In particular, E4-D requires the gate result to be written and exact iteration/terminal behavior; merely traversing a descriptor is insufficient.

C. Realization theorem

Theorem 1 (bounded composition under explicit obligations). Assume the fixed P_U discharges E4-D under its declared deterministic and serialized environment, and the gate basis on that same P_U satisfies E1–E3 with function g . Then, for every $d \in D_{64,512}$ and $x \in \{0, 1\}^{m(d)}$,

$$\text{Run}_{U,d}(\text{Enc}_U(d, x)) = (\text{OK}, \text{Eval}_g(d, x)) \quad (30)$$

after exactly $n(d)$ iterations. If the recognition boundary is additionally safety-sound for Safe, these traces also satisfy Safe.

Proof sketch. E4-D initializes the constant/input prefix. At iteration i , the descriptor bound makes both sources earlier cells. E3 supplies q_0 , E2 identifies the readout with g from that class, and E4-D writes that same readout only to the canonical destination while preserving the established prefix. Prefix induction yields Eval_g after $n(d)$ iterations; the exact-schedule clause supplies terminal OK. E1 binds the bit used by E2 and E4-D to a causal same-suffix state distinction; without E1 the functional induction would show only an ordinary bounded g -evaluator, not a state-mediated one. Safety follows only from the separate soundness premise. $\square$

The theorem decomposes local gate, reset, scheduling, and frame obligations; it is not a claim that tests enumerate the descriptor domain. It also does not prove E4-A, in which a compiler emits a different accepted BPF object per circuit.

IV. EBPF CALIBRATION CASE

A. Execution boundary

The program $P_U = \text{wm_circuit}$ is one fixed eBPF artifact with section $\text{SEC}(\text{syscall})$. In the recorded environment the in-kernel verifier accepts it and userspace executes it offline through the BPF_PROG_RUN interface; the retained host invocation uses `libbpf's bpf_prog_test_run_opts()` wrapper. Linux documents the execution interface and the `syscall` section/program-type mapping [10], [11]. No live hook is involved. The experiment is privileged and local, and its acceptance and resource facts remain specific to the recorded program type, kernel, and architecture.

The carrier boundary is:

The gate map $G0$ is a dedicated BPF_MAP_TYPE_HASH map with `max_entries = 2`. It is non-LRU and retains the default preallocation; Linux documents the entry bound, default preallocation, update flags, and success/negative-error convention for this map type [12]. The discipline requires successful reset and one external critical section over setup, invocation, and readback for every map in the footprint. The serial harness satisfies the no-interleaving use pattern, but the eBPF program implements no concurrency lock.

TABLE II
CARRIER BOUNDARY IN THE EBPF CALIBRATION CASE.

Host data and validation	Recognition unit	Post-acceptance interpretation
textual WMC1 $\rightarrow$ normalized map configuration $\text{Enc}_U(d, x)$	V accepts the fixed P_U , not WMC1 or d	P_U reads maps $\rightarrow \text{Sched}_U(d, x) \rightarrow$ helper returns and wire observations

B. Saturating-rank NAND

The gate uses pairwise-distinct keys: sentinel S and input keys A and B . Reset deletes all three and inserts S , establishing occupancy one. For the first input, zero updates S and one inserts fresh A ; for the second, zero updates S and one inserts fresh B . Proposition 2 states the abstract occupancy law. For the Linux instance, existing-key success, below-capacity fresh-key success, and at-capacity fresh-key failure are explicit premises restricted to valid arguments, default preallocation, successful reset, no interference, and the recorded Linux 6.17 environment. Reference [12] supplies the map interface and return convention; retained raw returns and mechanism controls instantiate the law for this object. The gate observes only the second update’s success predicate, not a portable error number.

The four executions are:

Proposition 2 (saturating-rank NAND). Let rank mean occupied-name count in a resource of capacity $k \geq 2$. Assume reset establishes rank $k - 1$, S is among those occupied names, and pairwise-distinct A and B are fresh. Existing-name update succeeds without changing rank; fresh-name update succeeds and increments rank below k , and fails without changing rank at k . Dispatch zero to S , the first one to A , and the second one to B . The second update’s success predicate is $\text{NAND}(a, b)$.

Proof. When $a = 0$, the first update preserves rank $k - 1$, so either second update succeeds. When $a = 1$, the first update raises the rank to k ; the second update succeeds for $b = 0$ because S already exists and fails for $b = 1$ because B is fresh. The outputs are therefore 1, 1, 1, 0. $\square$

NAND is functionally complete because $\neg x = \text{NAND}(x, x)$ and $x \wedge y = \neg \text{NAND}(x, y)$; the 512-gate descriptor bound limits circuit size, not the Boolean basis.

For the concrete Definition 1 witness, take $P = \text{wm_circuit}$ and let ℓ be the point in *circuit_step_cb* immediately before the second map update, after the first input-conditioned operation. Let $R_{G0}(\sigma)$ be the complete helper-relevant dynamic state of the dedicated map $G0$ —including key occupancy, buckets, preallocated element/free-list metadata, and any other map-local state read by the update helper—and set $\rho_{\text{obs}}(\sigma) = R_{G0}(\sigma)$. The occupied-key set $K_{G0}(\sigma)$ is only a derived projection of $R_{G0}(\sigma)$. Inputs $(0, 1)$ and $(1, 1)$ reach states σ_0 and σ_1 at ℓ with $K_{G0}(\sigma_0) = \{S\}$ and $K_{G0}(\sigma_1) = \{S, A\}$, hence $R_{G0}(\sigma_0) \neq R_{G0}(\sigma_1)$.

Take the common remaining suffix w to be the fresh- B update followed by the uniform capture and store of its success bit, and define $\text{Obs}(\tau) = [\text{ret}(\tau) = 0]$. The slice and ctx_w include the common program point, key B , value, $G0$ identity and static attributes (map type, capacity, default preallocation,

and BPF_ANY mode), relevant control and wire values, and every suffix-read component outside R_{G0} . The contract’s Env fixes the kernel, object, map instances, schedule, and absence of interference. Under the stated map-update contract, the two suffixes terminate with observations 1 and 0. The earlier input is not read by the suffix, and its persistent suffix-relevant effect is contained in the selected complete $G0$ dynamic state, so the non-selected contexts agree. Thus w is a conditional Definition 1 witness under this declared concrete-service contract. This is stronger than selecting occupancy alone and does not claim that the retained traces expose all internal kernel fields.

Under the gate discipline, take $s_{D_G}(\sigma) = (\text{phase}(\sigma), K(\sigma))$ on the invariant region X_{D_G} fixed by valid arguments, the dedicated preallocated map contract, successful reset, serialization, and no interference. Adequacy on this region is an explicit service-contract premise, not a machine-checked model of every kernel field. Phase ranges over the reset/sentinel/first/second-operation stages and $K \subseteq \{S, A, B\}$ with $|K| \leq 2$. The carrier, and hence its future-observation quotient, is finite. For E1, the two complete gate words for $(0, 1)$ and $(1, 1)$ have prefixes *updS* and *insA* and the common remaining suffix just defined. The complete reset-plus-gate word need not be causal because reset erases incoming differences; the witness is tagged by the internal frontier after the first operation.

For the basis take $P_G = P_U$, $\text{Adm}_G = X_{D_G}$, reset as delete S, A, B then insert S, G as the $S/A/B$ selector, and *observe* = [second return = 0]. The witness above discharges E1; Proposition 2 and the reset premises discharge E2–E3.

This mechanism adds no extensional Boolean expressiveness to eBPF. It is a calibration case showing that a documented stateful service can supply a reusable post-acceptance gate; it does not by itself establish report omission.

C. Fixed interpreter and map ABI

In addition to $G0$, *wm_circuit* uses *TAPE* and four interpreter maps:

- *TAPE* records build variant, capacity, selected raw return, and error count;
- *CIRCUIT* stores normalized records $(\text{op}, \text{src0}, \text{src1}, \text{dst})$;
- *WIRES* stores constants, primary inputs, and canonical SSA-style gate outputs;
- *VM_CONTROL* supplies the ABI version and declared counts;
- *VM_TRACE* stores per-gate validity, output, and, for state-mediated variants, the raw second-helper return.

The host parser checks and normalizes WMC1, writes map cells, and retains the requested output list. The list is

TABLE III
SATURATING-RANK NAND EXECUTIONS.

a	b	Operation word after reset	State before second operation	Second result	Output [$ret = 0$]
0	0	$updS; updS$	$\{S\}$	success	1
0	1	$updS; insB$	$\{S\}$	success	1
1	0	$insA; updS$	$\{S, A\}$	success	1
1	1	$insA; insB$	$\{S, A\}$	failure	0

projected only after $status = OK$ and is never read by BPF. The source-level mask is guarded by host control flow; the negative suite tests rejection status and execution counts, not an injected runtime masking path. Inside the kernel, a bounded loop handles at most 512 descriptors. Each iteration copies descriptor and source values before helper calls, resets $G0$, invokes the gate, and updates the canonical destination by key, avoiding retention of map-value pointers across those calls.

Canonical destinations prevent source/destination aliases and forward references. Covered malformed versions, counts, opcodes, destinations, or sources receive non-OK status. For valid descriptors, the topological restriction makes every source earlier. External whole-transaction serialization prevents concurrent mixed configurations; successive serial invocations overwrite shared cells, and stale physical cells may remain.

The verifier’s acceptance unit is the fixed interpreter artifact rather than each descriptor, although the harness may reload the same object for different datasets. Changing map configuration changes the interpreted circuit. This is the E4-D carrier distinction, not evidence for a compiler that emits a new accepted BPF object per circuit.

D. Source-level invariant and evidence boundary

Lemma 1 (source-level interpreter prefix invariant). Assume the update laws of Proposition 2, successful reset and required map/helper calls, the E4-D serialization environment, valid $Enc_U(d, x)$, and that the captured translated object preserves the manually inspected source control/data dependencies. For every integer t with $0 \leq t \leq n(d)$, after successful completion of the ordered callback indices $0, \dots, t-1$, every canonical wire below $2 + m(d) + t$ equals the corresponding prefix of $Eval_{NAND}(d, x)$.

Proof sketch. The outer program initializes constants and preserves host-written inputs. A callback validates the current opcode, destination, and earlier sources, then copies the descriptor and source bits. Proposition 2 gives NAND after reset; success writes only the canonical destination and increments the completed count. The outer program requests $n(d)$ iterations through bpf_loop ; under the helper contract, callback return zero continues, return one stops, and the helper reports the number of iterations performed [13]. The interpreter accepts only when both that return and its own completed count equal $n(d)$. Induction on t gives the prefix claim. This is a source-level argument supported by retained translated dumps for manual inspection, not a machine-checked eBPF-semantics proof. $\square$

The preserved object, verifier metadata, and loaded-program tag support E4-D’s fixed-acceptance unit. Source and translated dumps support initialization, ordered bpf_loop use, validation, reset, one-destination writes, and bounds; audits exercise outputs and negative status/count behavior. Serialization remains external because the program has no lock. Thus A is recorded, C has a conditional witness under the declared map-service/no-interference contract, and P is supported under additional source-to-object, frame, and serialization premises. For this `wm_circuit` carrier, nodes R and W are absent: no interpreter-specific Linux report-cell extractor or deployment policy is supplied.

V. EVALUATION

A. Recorded environment and primary run

The primary evidence is the run bundle `results/interpreter/interpreter-final-20260711-02/`. It records Ubuntu 24.04, Linux 6.17.0-35-generic on aarch64, and preserves four BPF variants, captured loaded-program metadata, verifier logs, descriptors, JSONL outputs, an author-run semantic audit, a then-current selected source/manuscript snapshot, and a self-issued SHA-256 manifest. The final manuscript postdates that run and is not claimed to be covered by its manifest. The bundle is author-generated and separately author-audited, not a third-party reproduction.

The dataset contains exactly

$$\begin{aligned}
 38,533 &= 26,488 \text{ per-gate records} \\
 &+ 12,037 \text{ successful run records} \\
 &+ 8 \text{ negative-control records.}
 \end{aligned} \tag{31}$$

These are heterogeneous evidence rows, not “38,533 tests.” In particular, the explicit arithmetic baseline’s gate records do not contain state-mediated helper-return traces.

B. Coverage

Named and random circuits exhaust their own finite inputs; the joint 64-input boundary uses only all-zero and all-one vectors, not 2^{64} inputs. The corpus is regression evidence, not an enumeration of $D_{64,512}$. Serial reuse detects common reset/stale-state regressions but is neither a concurrent stress test nor proof of the external critical-section premise.

C. Separate semantic audit and integrity check

The author-run auditor does not rely solely on the runner’s pass flag: it also reconstructs named and boundary descriptors, regenerates the seeded 100-DAG corpus, independently evaluates complete wire vectors, checks destinations, boundary

TABLE IV
RECORDED EVALUATION COVERAGE.

Dataset and input coverage	Successful runs	Per-gate records
9 named circuits, exhaustive per circuit	39	166
100 random DAGs, exhaustive per DAG (≤ 6 inputs, ≤ 24 gates; seed 3235823838)	1,876	23,776
512-gate deep and 64-input/512-gate/578-wire joint boundaries	4	2,048
zero-gate descriptor; 10,000 serial NAND/adder/mux invocations	10,001	0
capacity-64, forced-sentinel, and explicit-logic controls	117	498
8 malformed ABI/count/op/destination/reference cases	—	—

gates and eight malformed cases, and matches runtime tags to loaded-program metadata. It reports success. A subsequently written self-issued SHA-256 manifest also verifies; it detects drift but is not a signature, timestamp, attestation, or independent reproduction.

D. Mechanism attribution

Three variants separate the capacity mechanism from the truth table and artifact acceptance.

First, increasing capacity from 2 to 64 removes saturation in the tested schedule; all 166 named-corpus control gate outputs become 1. Second, forcing the second one-bit input to update S removes the fresh- B insertion; again all 166 outputs become 1. Third, an arithmetic baseline computes NAND without the capacity mechanism and produces the expected named-circuit results. All four variants—the state-mediated gate, two controls, and baseline—load and execute in the recorded environment.

The controls support mechanism attribution: saturation and the second fresh name are both necessary for the gate’s zero output. They do not show equivalent verifier reports. The baseline confirms that the mechanism gains no new Boolean function unavailable to ordinary bytecode.

E. Calibration result

The cited verifier overview states its objective in terms of determining program safety and validating paths, arguments, and memory access [14]; that overview itself does not specify a complete functional certificate for map-mediated computation. The retained `wm_circuit` evidence has no report extractor at its interpreter frontier, so we neither test nor infer how precisely a verifier cell tracks that program’s helper return. This carrier is a LangSec calibration of the boundary between recognized safety and interpreted functionality: it gives a conditional C witness under the declared service contract and supports P under additional premises, but it is not evidence of unsound acceptance and its interpreter-specific R node remains unestablished. Section 5.7 reassesses separate stock-Linux trace evidence; it does not evaluate R for the real Linux target from the V1 samples.

F. Fixed auxiliary executable report instance

To exercise the report-relative criterion without relabeling Linux verifier artifacts, we embed the auxiliary R instance in the full model carrier of Section 7.1 and fix

$$M_{linux_r_aux_v1} = (V_{linux_r}, I_{hash}, \text{Report}_{aux}, K_{obs}, P_{aux}, \ell, D, F, \emptyset, \emptyset, \emptyset, \emptyset). \quad (32)$$

P_{aux} is accepted by the custom report-producing recognizer V_{linux_r} ; it is not a BPF object accepted by the stock Linux verifier. I_{hash} is a finite, deterministic, serialized, no-interference semantics restricted to the non-evicting HASH-map update cases in the fixed program. Here Report_{aux} means only `report.json["report_cells"]`; `derivation.json` records `worklist/computation` provenance and is not part of the report-label interface. The last four components are the typed empty actor set, effect set, drive relation, and permitted-effect set; they embed the R instance in the common carrier and make no W claim.

Executable certificate. $R(M_{linux_r_aux_v1})$ holds (artifact status: established).

Certificate argument. The recognizer accepts P_{aux} . Exhausting $a \in \{0,1\}$ with $b = 1$ in the singleton serialized/no-interference environment gives $F = \text{Reach}_{I_{hash}}(P_{aux}, \ell) = \{\text{frontier:S}, \text{frontier:AS}\} \subseteq X_D$. Here X_D additionally contains the corresponding two terminal states. Let $a_{obs} \in A_D$ be the discipline action named `update-suffix-and-observe`, write c_{obs} for the exact encoded runtime-operation symbol `bpf_map_update_elem(G0, suffix_key, one, BPF_ANY); observe(ret==0)`, and set $\iota_{P_{aux}, \ell}(a_{obs}) = c_{obs}$; this singleton encoding is injective. Concrete execution of $\iota_{P_{aux}, \ell}(a_{obs})$ is defined on the two frontier states, undefined on the terminal states, and agrees with D on definedness, output, and successor. Thus the checker establishes operational adequacy on all of X_D . The observation contract fixes ρ_{obs} as the occupied-key set, `Obs` as the ordered success-bit word, `Slice` as the program-phase/service-context pair, and `Env` as that singleton environment; in particular, $\rho_{obs}(\text{frontier:S}) = \{S\} \neq \{S, A\} = \rho_{obs}(\text{frontier:AS})$. For every ordered pair in F^2 and every word in $\mathcal{W}_D(F) = \{\varepsilon, a_{obs}\}$, the checker establishes the remaining runtime-word, common-context, observer-compatibility, and K_{obs} -soundness obligations. These discharge every admissibility item except unique-cell coverage.

Report_{aux} emits one unique cell $a^\#$ that covers both frontier states, completing $\text{Adm}(P_{aux}, \ell, D, F; K_{obs})$. The exact finite future-observation quotient assigns them different classes, so $|\beta_D(F_{a^\#})| = 2$; moreover a_{obs} yields observations 1 and 0. Hence the same suffix first witnesses Definition 1 and the tagged tuple $(P_{aux}, \ell, D, F, a^\#, a_{obs})$ satisfies Definition 2. Therefore $R(M_{linux_r_aux_v1})$ holds. The evidence also records 21 domain/action *return-class containment* checks plus two report-cell *successor-containment* checks, all with zero violations; the former are not 21 full post-state checks. Each of

the four negative controls—exact occupancy tracking, capacity 64, forced sentinel, and an unobserved return—removes the R witness. $\square$

On the archived bundle, a separately implemented, author-run checker reconstructs reachability, unique-cell coverage, the quotient, and the factorization verdict without importing the model implementation, and emits the established auxiliary-R verdict. Separately, one regression test builds the same deterministic model bundle twice and byte-compares its five formal JSON files; this is a determinism test, not experimental evidence. The certificate is executable finite-model evidence, not a machine-checked proof. The retained kernel calibration has four oracle rows for the two assignments (0, 1) and (1, 1); it calibrates those two restricted service outcomes only. It proves no refinement or bisimulation between I_{hash} and Linux and extracts no stock-verifier cell. The auxiliary certificate therefore remains independent of the stock-Linux evidence reassessment below. Its bundle is `results/linux_r/linux-r-v1/`.

G. Stock-Linux trace evidence: V1 correction and V2 proof-bound result

This subsection analyzes a completed local stock-Linux experiment as retrospective trace evidence. The kernel capture is primary evidence that one prune event and two runtime samples occurred on the frozen tuple. The integrated and frozen-bundle checkers validate the legacy adapter’s declared evidence relationships offline; their regression suites are not substitutes for the capture, and neither layer converts the author-declared operational prune-report into a documented Linux functional-report contract. More importantly, one sample per case does not establish a stable must outcome for the actual target.

For notation, let M_{Linux} denote the actual stock-Linux/object/kernel target on the frozen V1 tuple. It is the target of the evidence-bounded query, but the V1 record does not provide a complete transition relation, Linux functional-report contract, or outcome-eligibility proof for it. It must not be identified with $M_K^{legacy} = M_{adapter}$ below, which is the author’s retrospective finite adapter construction.

The legacy adapter attempted to apply Definition 2 without reusing the auxiliary semantics by fixing the following hash-bound, evidence-restricted carrier:

$$M_K^{legacy} = M_{adapter} = (V_K, I_K, \text{Report}_K^{prune}, K_{obs}^K, P_R, \ell_K, D_K, F_K, \emptyset, \emptyset, \emptyset, \emptyset). \quad (33)$$

P_R is the fixed verifier-accepted XDP object `rac_single`, not the interpreter $P_U = \text{wm_circuit}$. V_K is the recorded stock Linux 6.17.0-35-generic verifier at exact-comparison level 0, and $\ell_K = \text{pre}(41)$ is the translated caller-side frontier before the common `shared_suffix` call. For this legacy retrospective construction, I_K is a manuscript-defined finite, phase-tagged, serialized relation constructed after the capture by restricting and phase-tagging the bundle’s two recorded executions. It is neither the bundle adapter’s self-loop transition system nor the set of all Linux executions. Its declared

input/environment domain contains exactly the two captured selector histories, so the following reachability equality is a definition of the restricted analysis carrier, not an empirical completeness claim about all states that Linux can reach:

$$\text{Reach}_{I_K}(P_R, \ell_K) = F_K = \{\sigma_0, \sigma_1\}. \quad (34)$$

Let $X_{D_K} = \{\sigma_0, \sigma_1, \sigma_0^+, \sigma_1^+\}$, where these symbols denote constructed, phase-tagged concretization witnesses for the two prefix cases and their post-suffix records. They are not direct snapshots of complete runtime registers, stack, and helper-internal map state. The two `runtime.json` final-state records bind only the derived G0 key-set projections $\{S, B\}$ and $\{S, A\}$ and their program-level outcomes; the verifier capture separately binds the selected State V2 records. Let a_B be the abstract insert- B action. Define a macro-symbol $c_B \in \Sigma_{op}(P_R)$ whose declared concrete interpretation executes the exact remaining translated suffix, and set $\iota_{P_R, \ell_K}(a_B) = c_B$. Fix the one-step discipline

$$\begin{aligned} S_{D_K} &= \{p_0, p_1, q_0^{\text{term}}, q_1^{\text{term}}\}, \\ A_{D_K} &= \{a_B\}, \quad O_{D_K} = \{0, 1\}, \\ O_{obs}^K &= \{\varepsilon, 0, 1\}, \\ s_{D_K}(\sigma_i) &= p_i, \quad s_{D_K}(\sigma_i^+) = q_i^{\text{term}}, \quad \delta_{D_K}(p_i, a_B) = q_i^{\text{term}}, \\ \lambda_{D_K}(p_0, a_B) &= 1, \quad \lambda_{D_K}(p_1, a_B) = 0. \end{aligned} \quad (35)$$

At the concrete level, the declared relation contains exactly

$$\sigma_i \xrightarrow{c_B/b_i} I_K \sigma_i^+, \quad (b_0, b_1) = (1, 0), \quad (36)$$

for this action and has no c_B -transition from σ_0^+ or σ_1^+ . These transitions are the manuscript wrapper; the bundle’s `runtime.json` records bind their key-set projections and program-level outcomes, not complete concrete endpoints. There are no outgoing actions from $q_0^{\text{term}}, q_1^{\text{term}}$. Both observers return bit words in O_{obs}^K : explicitly, $\text{Obs}_{D_K}(\varepsilon) = \varepsilon$ and $\text{Obs}_{D_K}(b_i) = b_i$; on the corresponding declared concrete side, Obs maps the empty trace to ε and each recorded c_B trace to b_i . Thus observation compatibility covers both words in $\mathcal{W}_{D_K}(F_K) = \{\varepsilon, a_B\}$. The phase tags and displayed transitions make operational adequacy and runtime-word inclusion true by construction for this restricted relation. They do not independently validate a larger Linux transition semantics.

The stock-specific contract K_{obs}^K sets $\rho_{obs}^K(\sigma) = R_{G0}(\sigma)$, the complete helper-relevant dynamic G0 state defined in Section 4.2, and observes the program-level success-bit word. The recorded key set is only a derived projection K_{G0} : $K_{G0}(\sigma_0) = \{S\}$ and $K_{G0}(\sigma_1) = \{S, A\}$, so $R_{G0}(\sigma_0) \neq R_{G0}(\sigma_1)$ even though the records do not expose every internal field. The context fixes the caller phase and suffix, kernel/object/program and map identity, key/value constants, map type/capacity/flags, serialization, the single-artifact condition, and every suffix-read component outside R_{G0} . The checker compares the seven declared normalized runtime context fields. Separately, a manuscript read-set review covers the caller call/return path at translated PCs 41–44 and the

shared_suffix callee at PCs 107–122. The callee overwrites its key/value working registers and stack slots; outside the helper it otherwise reads only fixed constants and the fixed G0 map identity, while the caller merely invokes the callee and returns its result. Map-local buckets, element/free-list metadata, and any other helper-read G0 state belong to R_{G0} ; the singleton hash-bound Env fixes the kernel, object, exact map instance, service/allocation choices, schedule, and absence of interference. The two records therefore have the same non-selected suffix-read context. Since the selected-state projections already differ, the contract’s equal- R_{G0} premise on F_K relates only an identical state to itself; determinism of the declared one-step service relation gives soundness. The recorded success bits equal Obs_{D_K} , giving observer compatibility. Thus the runtime-word, common-context, operational-adequacy, observation-compatibility, and soundness premises are discharged on this explicitly finite carrier, not on general Linux execution. The read-set argument and the typing of $I_K, D_K, K_{\text{obs}}^K$ are manuscript-reviewed obligations, not claims about what the checker machine-verifies.

$\text{Report}_K^{\text{prune}}$ is an explicitly author-declared *operational prune-report* selected to study the captured event: it assigns an admitted prefix witness to the retained verifier-state representative of a successful `states_equal` comparison that actually causes `is_state_visited` to prune the current state. The directed prune edge is the membership relation; we do not reinterpret `states_equal` as a symmetric mathematical equivalence over complete concrete states. The cited verifier overview describes the verifier as a safety mechanism; that overview itself does not specify this interface as a complete functional report over runtime map occupancy. The report is therefore an analysis projection extracted from concrete prune events, not a Linux-specified certificate whose intended semantics the experiment has falsified.

The frozen manifest binds the kernel release, BTF and configuration hashes, object SHA-256, loaded program id/tag/pin, and translated-bytecode SHA-256. The selected fexit event records both `states_equal_success=true` and `is_state_visited_prune=true` at instruction 41. Its retained and current histories are distinct and pass through the translated branch calls for $a = 1$ and $a = 0$, respectively, before reaching the same frontier. Their state hashes also differ (516c47f044cc3fc3 versus a66d912d58c91de4); membership follows the observed directed prune edge, not hash equality. A reviewed path-correlation checker binds both histories to that frontier and to the same remaining translated suffix; the capture-completion record reports zero lost events and zero parse errors.

Normalized membership checks associate the constructed witness σ_0 with the current captured verifier state and σ_1 with the retained captured verifier state; they do not capture complete runtime frontier states. The runtime records bind the witnesses’ derived G0 key-set projections to $\{S\}$ and $\{S, A\}$. Within the restricted exact-0 State V2 shape and the declared

operational report, the shape and unique-cell checkers establish

$$\gamma_{\ell_K}^K(\text{retained:516c47f044cc3fc3}) \cap F_K = \{\sigma_0, \sigma_1\}. \quad (37)$$

Neither state belongs to any other extracted report cell on F_K . Hence

$$\pi_R(\sigma_0) = \pi_R(\sigma_1) = \text{retained:516c47f044cc3fc3}. \quad (38)$$

Under the jointly defined word a_B , the recorded *program-level success bits* are 1 from σ_0 and 0 from σ_1 ; they are not raw `bpf_map_update_elem` returns (0 on success and negative on failure). The legacy adapter copies those observations into a deterministic relation, so it obtains $p_0 \not\sim_{D_K} p_1$ and a factorization failure in $M_K^{\text{legacy}} = M_{\text{adapter}}$. That legacy-model result is not a sound substitute for the premise that the corresponding M_{Linux} outcomes are fixed must outcomes.

The evidence-bounded reassessment imports only the V1 identity, runtime, and operational-prune premises and deliberately excludes the legacy factorization, quotient, Definition 2, and verdict outputs. It finds outcome eligibility = NOT_ESTABLISHED because V1 records one trial for each case and the capture harness uses `repeat = 1`; neither the declared environment nor the adapter closes the possible outcome set. Hence the exact frozen operational-prune query has verdict UNKNOWN. An attempted NONFACTORING assessment fails witness-not-outcome-eligible; a broader-runs query remains UNKNOWN; and a query asking for a Linux-specified functional report is OUT_OF_SCOPE because no such report contract has been identified.

Therefore this paper does *not* claim $R(M_{\text{Linux}})$, a Definition 2 instance for M_{Linux} , or a trace-local R certificate. The factorization result belongs only to the historical construction $M_K^{\text{legacy}} = M_{\text{adapter}}$. STOCK_LINUX_R_ESTABLISHED_FOR_FROZEN_TUPLE remains the historical output of a byte-frozen legacy adapter; it means only that its encoded gates replay on the retained files. The V1 record is carrier-, tuple-, observer-, and suffix-specific evidence for an observed prune edge and a MAY-difference, not for a stable residual-language witness. It does not generalize to uncaptured inputs, continuations, or reachable states; other kernels, configurations, BTFs, compiler outputs, or eBPF objects; general `states_equal` semantics; verifier unsoundness; a vulnerability; P; W; or a weird machine. The frozen evidence is under `residuality-auditor/stock-linux-r-proof/`; the active supersession rule is the evidence-bounded reassessment stated here and enforced by the public tests.

V2 is a separate prospective experiment, not a reinterpretation of the V1 tuple. Let $M_{\text{Linux}}^{\text{V2}}$ denote the controlled stock-Linux/object/kernel/map instance generated by `residuality-auditor/linux/scripts/run_stock_r_v2.sh` for the accepted `rac_v2` witness. The V2 contract pre-commits an outcome-free verifier-prune query, an invocation-scoped fentry/fexit capture, an array-map witness, a fixed source/build closure, and a proof checker. Its runner seals the runtime identity first, then writes `proof/must-outcome-proof.json` and `proof/history-case-binding.json`, then audits the bundle. The proof document binds

the query digest, source and build closure, object SHA-256, translated-bytecode digest, BTF/kernel identifiers, runtime-event identity, checker calculus, and checker source closure. The binding independently joins the retained/current history digests to proof cases 0/1 and requires one frontier, report cell, common suffix, observer, and exact-scope digest.

The V2 must-outcome proof closes the modal gap that made V1 fail closed, while the history-case binding closes the join from that proof to the operationally selected histories. The checker does not accept runtime divergence as its own proof. It replays the proof’s two case derivations in the calculus `stock-r-v2-array-map-must-outcome-v1`: the low input bit determines the array-map slot-0 update, the subsequent lookup reads slot 0, and the program returns that low bit. The derived case map is therefore $\{0 \mapsto 0, 1 \mapsto 1\}$. A bundle is accepted only when these derived outcomes agree with the recorded repeated runtime outcomes, the observed prune is the declared operational-prune event, the history-case join verifies, and all identity receipts match the sealed manifest. A valid proof without the binding remains UNKNOWN/NOT_ESTABLISHED; absent or malformed proof, identity, binding, or runtime evidence fails closed.

Under those gates, one fresh privileged run on stock Ubuntu kernel 6.17.0-35-generic has the positive evidence-bounded result: `outcome_eligibility.status = ESTABLISHED`, `outcome_eligibility.method = MUST_OUTCOME_PROOF_WITH_HISTORY_CASE_BINDING`, `assessment.status = NONFACTORING`, and `assessment.scope = EXACT_STOCK_R_V2_QUERY`. Its certificate is `NONFACTORING@1d5f86d80494575c23f539248614105559dd15380c580d0d2388c24941b6d255`. This is a report-factorization result for the declared operational-prune report on the controlled V2 witness. It still does not establish a Linux-specified functional report, general `states_equal` semantics, other inputs or continuations, other kernels or objects, verifier unsoundness, a vulnerability, P, W, or a weird machine. It also does not retroactively repair V1: the frozen V1 exact query remains UNKNOWN.

The U4 reference layer separates source validation from generic authorization. Source-specific adapters validate and compile V1/V2 bytes into a versioned claim, typed evidence graph, and proof DAG; the generic checker validates node digests, graph order and dependency, proof references, and the finite exact-claim rules without reading the bundle’s terminal verdict. On retained inputs it returns V1 BLOCKED/INCONCLUSIVE, whose strongest profile contains only MAY_OUTCOME and REPORT_COLLISION, and V2 CERTIFIED/NONFACTORING at quantifier AT, authority OPERATIONAL_OBSERVATION, and grade OUTCOME_FREE_PRECOMMITTED. A retained hostile matrix changes one dimension at a time: five requested FORALL, specified-report, observer, suffix, and TRANSPORTED lifts are blocked; proof-wide FORALL, specified-report, transported-grade, and report-relation rewrites plus outcome-to-selector dependency, payload

tampering, and an absent proof premise are invalid. This is an executable exact-scope claim-discipline control, not typed scope transport, family coverage, compiler correctness, or a new foundational theorem.

CRL adds one guarded transport theorem at the verifier-contract layer. In the abstract contract $V = (S, H, \pi, equal, visited, step, \eta, \rho)$, $StockR_V(s, w)$ contains two source histories in one report cell with singleton, observer-distinct outcomes under one continuation. If an admitted context term C is transparent—total instruction correspondence on the witness region, footprint/effect disjointness, collision, suffix, must-outcome, observer, frontier, report-cell and history-map preservation, a target-conformance bridge, outcome-independent selection, and no target terminal verdict premise—then $StockR_V(t, C(w))$ holds for the exact target scope. The executable certificate is a DERIVED_CONTEXTUAL chain whose source-claim, transform, and target-claim digests are recomputed by the checker; the underlying EBRC proof rule remains CONTEXT_TRANSPORT, and the only authorized target claim is AT(target) with evidence grade TRANSPORTED.

The retained U6 VM matrix instantiates this theorem over the frozen BOUNDED_CONTEXT_SUITE_ONLY suite. The published run `contextual-matrix-live-20260720-03` contains twelve expected cases: six transparent contexts certify and six hostile or missing-obligation contexts fail closed. The replay capsule selects two transparent targets, `transparent.xor.depth1` and `transparent.add-mul.depth2`; their target digests and certificates are:

```
transparent.xor.depth1
xlated digest: c902fec11d3825fa1317fab76053127
95d4531d83e26aa653fcc214632217de
object hash: 84c0ba6fc0d1a702dded1037f7e782d
51d3713b54221e67db4474e0aaf6ac531
certificate: NONFACTORING@23b72c129e12520d
f1b05580f4ab74582f49b4e4f442
db3e05e207a7deffc1e2
transparent.add-mul.depth2
xlated digest: e36423e7fcf712ee187636fd0dbe3a91
fafdbd82b85f60b6ee4cbe85df49adad
object hash: 13902d95bac31c2fe861dccc24a58b5e3
b4d43f7d4b90a0165bf85a1310c7a02
certificate: NONFACTORING@1d19c14f69a1866
48acae58c57c68faf7b9719a51
b2e870e53bafb81efc663
```

The target translated-bytecode digests are both distinct from the V2 source digest and from each other; each runtime bridge is VERIFIED over four trials. Each contextual hostile matrix reports `all_expected = true` with three blocked unsupported claims and nine invalid graphs, including stale or rewritten DERIVED_CONTEXTUAL chains. These are target-bound transport instances, not a FORALL context theorem, compiler-correctness proof, Linux functional report, verifier vulnerability, P, W, or weird-machine result.

H. Threats to validity

The experiment uses one kernel build and architecture. The argument relies on a dedicated preallocated non-LRU

map, successful reset, distinct keys, the stated update laws, and whole-map-set mutual exclusion. Gate-emitting datasets preserve raw returns; the 10,000-run stress file records run/status/output evidence without per-gate raw rows. The proof uses only success versus failure and promises no portable error number.

Finite testing does not prove all $D_{64,512}$. Theorem 1 instead depends on the stated source-level initialization, validation, ordered iteration, gate, and frame obligations; the corpus seeks counterexamples. Translated dumps and verifier logs are retained for manual inspection and manifest hashing, not consumed by an automated data-flow checker. Status masking has a source-order guard, while the negative suite tests rejection/status rather than an injected masking path. There is no machine-checked eBPF-semantics proof, and production-verifier safety soundness is assumed only for the optional Safe conclusion.

The stock-Linux evidence is author-generated and author-reviewed on isolated kernel/object tuples; it has not been independently reproduced. For V1, the operational report, two-history carrier, observer, and suffix were finalized retrospectively, so the record is vulnerable to selection effects even though the internal checks pass. V1 binds BTF, configuration, and source-map metadata but does not preserve the target `verifier.c` function bodies; its restricted shape lemma is not a source-level theorem about general Linux `states_equal` behavior. For V2, the prospective runner, must proof, and history-case binding remove the specific missing-outcome and witness-join gaps, but the proof calculus is a small source-level microsemantic checker, not a machine-checked semantics for Linux, libbpf, JIT code, helper internals, or verifier source. CRL adds a model-contract transport theorem and two generated target instances, but it still trusts the source-specific adapters and the stated target-conformance bridge. The generic checker validates only the compiled finite fragment; source-specific adapters remain trusted for interpreting the original bundle bytes. Treating a raw event, a state hash, a legacy adapter verdict, repeated runtime divergence without proof, bytecode similarity without a bridge, or similar verifier-log text alone as R would not satisfy Definition 2.

VI. RELATED WORK

Language-theoretic security supplies the recognition/interpretation framing [1]–[4]; object-centric tracing reconstructs state-dependent low-level operation languages [15]. Weird-machine work studies unintended computation and exploitability [16], [17], insecure compilation makes the policy boundary explicit [18], and proof-carrying-code work identifies shadow execution outside a proof model [19]. Valid-input computation also appears in well-formed RPM metadata, while vulnerability-flow types derive abstract weird machines [20], [21]. Unlike work that demonstrates rich behavior, our criterion asks whether a declared computed report actually factors a future-observation quotient at one accepted-artifact frontier.

Abstract interpretation supplies concrete/abstract, soundness, and completeness vocabulary [5], [22]; our criterion is limited to uniquely assigned cells actually computed at one frontier. MOAT uses MPK to isolate potentially malicious BPF programs in response to verifier-based safety limitations [23], while mismorphism compares interpretations [24]. For eBPF, range-analysis verification and state embedding target verifier-logic soundness/coverage errors [25], [26]. VEP’s “programmability” means reducing verification-toolchain restrictions, while Rex addresses rejection-side language-verifier mismatch by replacing a separate static verifier with language-based safety and an extralingual runtime [27], [28]; P here instead means one fixed stock-verifier-accepted artifact interpreting a bounded family. DRACO checks functional specifications after verifier acceptance, and BPFVERIFY translates eBPF bytecode to bit- and memory-precise Horn clauses for functional verification [29], [30].

Recent preprints further separate the boundary: Heimdall treats higher-level defects surviving compilation and acceptance; Yaksha-Prashna extracts third-party bytecode behavior; bpfix localizes proof loss in rejected programs [31]–[33]. Trust-boundary semantic-gap work studies insufficient assertions after correctly implemented syntactic acceptance [34]. They do not jointly distinguish the obligations in our claim graph. Conversely, our `wm_circuit` interpreter carrier does not establish R or W; the auxiliary tuple establishes R only on its own carrier; V1 stock-Linux remains UNKNOWN; and V2 establishes only an exact-query operational-prune NONFACTORING result. These four 2026 works are cited as preprints, not peer-reviewed publications.

VII. LIMITATIONS, IMPLICATIONS, AND OUTLOOK

A. The claim graph has strict separations

To put all five nodes on one carrier, fix a model

$$M = (V, I, \text{Report}, K_{\text{obs}}, P_*, \ell_*, D_R, F_R, \mathcal{A}, \mathcal{E}, \text{Drive}, \text{Pol}), \quad (39)$$

enriched with named intended-semantics, safety, report-abstraction, and granularity contracts against which the evidence predicates below are evaluated, where $\mathcal{A}$ and $\mathcal{E}$ are actor and effect sets, $\text{Drive} \subseteq \mathcal{A} \times \mathcal{E}$ records effects actors can drive through the designated P_* , and $\text{Pol} \subseteq \mathcal{E}$ is the permitted-effect set. Define $A(M)$ by $P_* \in L_V$; $C(M)$ by $\exists \ell, \exists w. (P_*, \ell, w) \in L_{\text{causal}}(V, I; K_{\text{obs}})$; $P(M)$ when P_* is one fixed accepted bounded interpreter with a same-artifact basis $(P_*, \text{reset}, G, \text{observe}, D_G, \text{Adm}_G)$ satisfying E1–E3, P_* discharges E4–D using that basis, and g together with constants $\{0, 1\}$ is functionally complete for Boolean circuits; $R(M)$ when $\text{Adm}(P_*, \ell_*, D_R, F_R; K_{\text{obs}})$ holds and some $(P_*, \ell_*, D_R, F_R, a^\#, w)$ is output-witnessed report-relative residual on exactly that tuple; and $W(M)$ when $\exists a \in \mathcal{A}, \exists e \in \mathcal{E}. (a, e) \in \text{Drive} \wedge e \notin \text{Pol}$.

For classification, let $\text{Link}(M)$ require the R witness (when used) to lie in the gate/interpreter execution family establishing P and the W effect to arise by an actor driving that same encoded computation; unrelated co-resident witnesses do not

qualify. Let $\text{Unint}(M)$ mean that this programmable interpretation/effect is outside the boundary's intended semantics. Define

$$\text{WM}_{\text{policy}}(M) = P(M) \wedge W(M) \wedge \text{Link}(M) \wedge \text{Unint}(M). \quad (40)$$

Let $\text{Doc}(M)$ mean that this linked behavior uses documented semantics and preserves the declared safety contract, $\text{Conf}(M)$ mean that the report implementation conforms to its specified abstraction, and $\text{Gran}(M)$ mean that its R collision is forced by the declared report granularity. Define the narrower class by

$$\text{WM}_{\text{shape}}(M) = \text{WM}_{\text{policy}}(M) \wedge R(M) \wedge \text{Doc}(M) \wedge \text{Conf}(M) \wedge \text{Gran}(M). \quad (41)$$

Proposition 3 (non-implications among claim nodes). Over this model class, the following implications are invalid:

$$\begin{aligned} A \not\Rightarrow C, \quad C \not\Rightarrow P, \quad C \not\Rightarrow R, \quad P \not\Rightarrow R, \\ R \not\Rightarrow P, \quad P \not\Rightarrow W, \quad R \not\Rightarrow W. \end{aligned} \quad (42)$$

The definitions validate $R \Rightarrow C \Rightarrow A$ and $P \Rightarrow C \Rightarrow A$: Definition 2 contains a Definition 1 witness, and E1 uses the E4-D artifact.

Proof by finite countermodels. For $A \not\Rightarrow C$, accept only *skip*. For $C \not\Rightarrow P$ and $R \not\Rightarrow P$, use states $\{0, 1, \perp\}$, a one-shot bit read, and one report cell. Exact future-class cells give $C \not\Rightarrow R$; an E1–E4-D NAND interpreter with that exact partition gives $P \not\Rightarrow R$. Setting $\text{Pol} = \mathcal{E}$ gives $P \not\Rightarrow W$ and, in the one-shot model, $R \not\Rightarrow W$. Finally, an actor-driven, unintended, policy-excluded effect from the same encoded P family with an exact report partition gives $\text{WM}_{\text{policy}} \wedge \neg R$. $\square$

Thus bare C, abstract-interpretation incompleteness, or acceptance incompleteness does not imply P or a weird-machine classification.

B. Defensive implications

Audit the recognized property and report separately, enumerate actor-driven operations and environment assumptions, then test report factorization. Contract violations call for implementation fixes; documented collisions call for report refinement, restriction, or isolation only when the report intended that relation.

C. Weird-machine status and a future shape theorem

The `wm_circuit` carrier supports P under its source/object and serialization premises but establishes neither R nor W. The frozen V1 M_{Linux} record has a separate $M_K^{\text{legacy}} = M_{\text{adapter}}$ finite-model factorization result, but the current assessment of the V1 exact query is UNKNOWN, not R. The V2 $M_{\text{Linux}}^{\text{V2}}$ record establishes proof-bound NONFACTORING only for EXACT_STOCK_R_V2_QUERY; its report carrier, object, and witness are still not the interpreter carrier. The auxiliary $M_{\text{linux}_r_{\text{aux}}_{\text{v1}}}$ establishes R only for its fixed custom report and restricted service semantics. Because the auxiliary and V2 R carriers are different accepted artifacts and report carriers from the interpreter's P certificate,

Link forbids combining them. No evaluated model therefore establishes $\text{WM}_{\text{policy}}$ or WM_{shape} . A shape theorem must derive **macro closure** of acceptance and **report embedding** of the collision on one carrier; context sensitivity can defeat either. Complete-shell theory [22] may characterize report refinement, but cannot create reachability, reset, routing, linkage, or policy violation.

VIII. CONCLUSION

At a layered LangSec boundary, artifact recognition, post-acceptance interpretation, report factorization, bounded programmability, and policy-level weird-machine status are distinct: $R \Rightarrow C \Rightarrow A$ and $P \Rightarrow C \Rightarrow A$, while Proposition 3 gives finite countermodels for the other listed implications. The `wm_circuit` carrier records A, conditionally witnesses C, and supports P for a fixed 64-input/512-gate NAND interpreter, but establishes neither R nor W. The auxiliary tuple independently establishes R for its custom report. Stock-Linux V1 supplies a real exact-level-0 prune event and differing common-suffix samples, but without a must-outcome proof its evidence-bounded operational-prune query remains UNKNOWN. Stock-R V2 supplies a proof and history-case binding for a separate controlled witness and establishes NONFACTORING only for EXACT_STOCK_R_V2_QUERY; the generic checker preserves that exact boundary and rejects the tested claim lifts. CRL transports that exact source certificate to two separate generated targets, but only as exact `AT(target)` certificates, not as a family theorem. Neither the auxiliary, V2, nor CRL carrier supplies P or W or links to the interpreter, so the evidence establishes no WM_{shape} , policy-level weird machine, verifier unsoundness, vulnerability, or universal necessity theorem.

ETHICS AND DATA AVAILABILITY

Experiments run in an isolated local VM and use no third-party target or production data path. The interpreter runs attach to no live kernel hook. The stock-Linux capture briefly attaches an fexit observer to verifier-internal functions during an isolated program load; it observes verifier decisions without changing the accepted program's execution. No experiment attempts corruption, verifier bypass, or privilege escalation.

The repository is <https://github.com/Emtanling/eBPF-machine>; immutable eBPF-interpreter evidence is at commit 4309069a, under `results/interpreter/interpreter-final-20260711-02/`. The auxiliary report-instance evidence is at commit f665b1a, under `result/s/linux_r/linux-r-v1/`. The stock-Linux V1 trace evidence is published in the tagged V1.0 snapshot under `residuality-auditor/stock-linux-r-proof/`. Its `MANIFEST.json`, `CHECKSUMS.sha256`, embedded input hashes, raw capture, normalized certificates, proof outputs, and checker sources are co-versioned in that public snapshot. The bundled standard-library verifier emits `FROZEN_PROOF_BUNDLE_VERIFIED`. The prospective Stock-R V2, EBRC U4, and CRL U5/U6 sources and tests are under `residuality-auditor/linux/, residualit`

y-auditor/src/residuality_auditor/, and residuality-auditor/tests/; maketest-ebrc runs the focused generic controls, maketest-ebrc-context runs the contextual controls, and maketest-stock-r-tools passes 172 tests in the recorded VM environment. The public replay capsule is residuality-auditor/artifact/evidence/replay-capsule.tar.xz, SHA-256 3df6b96e3dded26e9f876db8f607278bc0a65a6df31b297cb6bd3043f44151f7, size 2,208,232 bytes. makereproduce-paper verifies the capsule and compares replayed V1/V2/CRL results against residuality-auditor/artifact/expected-results.json; makecontextual-matrix-live reruns the privileged BOUNDED_CONTEXT_SUITE_ONLY VM matrix in a fresh output directory. The prepublication archive residuality-auditor-v0.3.0-full.tar.gz had SHA-256 5fd0a2812c8c8db2fe5508440934817c1cf9293ba0c5df31317e8b38d94a90ec; V1.0 publishes the frozen proof payload directly rather than duplicating the archive. Any frozen-byte change requires a new version, updated checksums, and re-verification.

ACKNOWLEDGMENT AND AI-ASSISTANCE DISCLOSURE

OpenAI Codex assisted drafting and language revision throughout the manuscript, with substantive author-directed revision of the Abstract, Introduction, Sections 5.6–5.8, and Conclusion, plus artifact code/test revision and checks. The author independently reviewed the claims, proofs, citations, changes, and results and takes full responsibility. No conflict of interest is declared.

REFERENCES

- [1] L. Sassaman, M. L. Patterson, S. Bratus, and M. E. Locasto, “Security Applications of Formal Language Theory,” *IEEE Systems Journal*, vol. 7, no. 3, pp. 489–500, 2013, doi: 10.1109/JSYST.2012.2222000.
- [2] F. Momot, S. Bratus, S. M. Hallberg, and M. L. Patterson, “The Seven Turrets of Babel: A Taxonomy of LangSec Errors and How to Expunge Them,” in *2016 IEEE Cybersecurity Development (SecDev)*, pp. 45–52, 2016, doi: 10.1109/SecDev.2016.019.
- [3] L. Sassaman, M. L. Patterson, and S. Bratus, “A Patch for Postel’s Robustness Principle,” *IEEE Security & Privacy*, vol. 10, no. 2, pp. 87–91, 2012, doi: 10.1109/MSP.2012.31.
- [4] S. Ali, P. Anantharaman, Z. Lucas, and S. W. Smith, “What We Have Here Is Failure to Validate: Summer of LangSec,” *IEEE Security & Privacy*, vol. 19, no. 3, pp. 17–23, 2021, doi: 10.1109/MSEC.2021.3059167.
- [5] P. Cousot and R. Cousot, “Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints,” in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL ’77)*, pp. 238–252, 1977, doi: 10.1145/512950.512973.
- [6] A. Nerode, “Linear Automaton Transformations,” *Proceedings of the American Mathematical Society*, vol. 9, no. 4, pp. 541–544, 1958, doi: 10.1090/S0002-9939-1958-0135681-9.
- [7] G. H. Mealy, “A Method for Synthesizing Sequential Circuits,” *Bell System Technical Journal*, vol. 34, no. 5, pp. 1045–1079, 1955, doi: 10.1002/j.1538-7305.1955.tb03788.x.
- [8] G. Amato and F. Scozzari, “Observational Completeness on Abstract Interpretation,” *Fundamenta Informaticae*, vol. 106, nos. 2–4, pp. 149–173, 2011, doi: 10.3233/FI-2011-381.
- [9] F. Ranzato and F. Tapparo, “Generalized Strong Preservation by Abstract Interpretation,” *Journal of Logic and Computation*, vol. 17, no. 1, pp. 157–197, 2007, doi: 10.1093/logcom/exl035.

- [10] Linux Kernel Documentation, “Running BPF Programs from Userspace,” [Online]. Available: Linux v6.17 BPF documentation (accessed Jul. 11, 2026).
- [11] Linux Kernel Documentation, “Program Types and ELF Sections,” [Online]. Available: Linux v6.17 libbpf documentation (accessed Jul. 11, 2026).
- [12] Linux Kernel Documentation, “BPF_MAP_TYPE_HASH, with PER-CPU and LRU Variants,” [Online]. Available: Linux v6.17 map documentation (accessed Jul. 11, 2026).
- [13] Linux Kernel Developers, “bpf_loop Helper API,” *include/uapi/linux/bpf.h*, Linux v6.17. [Online]. Available: Linux v6.17 source (accessed Jul. 13, 2026).
- [14] Linux Kernel Documentation, “eBPF Verifier,” [Online]. Available: Linux v6.17 verifier documentation (accessed Jul. 11, 2026).
- [15] I. Palmer, E. Rogers, and R. Adams, “Object-Centric Tracing for Language-Theoretic Security in Low-Level Interfaces,” in *Twelfth Workshop on Language-Theoretic Security (LangSec 2026)*, 2026. [Online]. Available: langsec.org (accessed Jul. 12, 2026).
- [16] S. Bratus, M. E. Locasto, M. L. Patterson, L. Sassaman, and A. Shubina, “Exploit Programming: From Buffer Overflows to Weird Machines and Theory of Computation,” *USENIX ;login:*, vol. 36, no. 6, pp. 13–21, 2011. [Online]. Available: USENIX (accessed Jul. 19, 2026).

- [17] T. Dullien, “Weird Machines, Exploitability, and Provable Unexploitability,” *IEEE Transactions on Emerging Topics in Computing*, vol. 8, no. 2, pp. 391–403, 2020, doi: 10.1109/TETC.2017.2785299.
- [18] J. Paykin et al., “Weird Machines as Insecure Compilation,” arXiv:1911.00157, 2019, doi: 10.48550/arXiv.1911.00157.
- [19] J. Vanegue, “The Weird Machines in Proof-Carrying Code,” in *2014 IEEE Security and Privacy Workshops*, pp. 209–213, 2014, doi: 10.1109/SPW.2014.37.
- [20] S. Ali, M. E. Locasto, and S. W. Smith, “Weird Machines in Package Managers: A Case Study of Input Language Complexity and Emergent Execution in Software Systems,” in *2024 IEEE Security and Privacy Workshops (SPW)*, pp. 169–179, 2024, doi: 10.1109/SPW63631.2024.00021.
- [21] M. Lesani, “Vulnerability Flow Type Systems,” in *2024 IEEE Security and Privacy Workshops (SPW)*, pp. 157–168, 2024, doi: 10.1109/SPW63631.2024.00020.
- [22] R. Giacobazzi, F. Ranzato, and F. Scozzari, “Making Abstract Interpretations Complete,” *Journal of the ACM*, vol. 47, no. 2, pp. 361–416, 2000, doi: 10.1145/333979.333989.
- [23] H. Lu, S. Wang, Y. Wu, W. He, and F. Zhang, “MOAT: Towards Safe BPF Kernel Extension,” in *33rd USENIX Security Symposium*, pp. 1153–1170, 2024. [Online]. Available: USENIX (accessed Jul. 19, 2026).
- [24] P. Anantharaman et al., “Mismorphism: The Heart of the Weird Machine,” in *Security Protocols XXVII*, LNCS 12287, pp. 113–124, 2020, doi: 10.1007/978-3-030-57043-9_11.
- [25] H. Vishwanathan, M. Shachnai, S. Narayana, and S. Nagarakatte, “Verifying the Verifier: eBPF Range Analysis Verification,” in *Computer Aided Verification*, Lecture Notes in Computer Science, vol. 13966, pp. 226–251, 2023, doi: 10.1007/978-3-031-37709-9_12.
- [26] H. Sun and Z. Su, “Validating the eBPF Verifier via State Embedding,” in *18th USENIX Symposium on Operating Systems Design and Implementation*, pp. 615–628, 2024. [Online]. Available: USENIX (accessed Jul. 19, 2026).
- [27] X. Wu et al., “VEP: A Two-stage Verification Toolchain for Full eBPF Programmability,” in *22nd USENIX Symposium on Networked Systems Design and Implementation*, pp. 277–299, 2025. [Online]. Available: USENIX (accessed Jul. 19, 2026).
- [28] J. Jia et al., “Rex: Closing the Language-Verifier Gap with Safe and Usable Kernel Extensions,” in *2025 USENIX Annual Technical Conference*, pp. 325–342, 2025. [Online]. Available: USENIX (accessed Jul. 19, 2026).
- [29] D. Lu, B. Tang, M. Paper, and M. Kogias, “Towards Functional Verification of eBPF Programs,” in *Proceedings of the 2024 ACM SIGCOMM Workshop on eBPF and Kernel Extensions (eBPF ’24)*, pp. 37–43, 2024, doi: 10.1145/3672197.3673435.
- [30] M. Bromberger, S. Schwarz, and C. Weidenbach, “Automatic Bit- and Memory-Precise Verification of eBPF Code,” in *Proceedings of the 25th Conference on Logic for Programming, Artificial Intelligence and Reasoning*, EPIc Series in Computing, vol. 100, pp. 198–221, 2024, doi: 10.29007/sj41.
- [31] V. A. Dasu, M. Santra, M. R. U. Rashid, A. Kumar, S. Tizpaz-Niari, and G. Tan, “Heimdall: Formally Verified Automated Migration of Legacy eBPF Programs to Rust,” arXiv:2605.25411, 2026, doi: 10.48550/arXiv.2605.25411.
- [32] A. Singh et al., “Yaksha-Prashna: Understanding eBPF Bytecode Network Function Behavior,” arXiv:2602.11232, 2026, doi: 10.48550/arXiv.2602.11232.
- [33] Y. Zheng et al., “Characterizing and Bridging the Diagnostic Gap in eBPF Verifier Rejections,” arXiv:2607.02748, 2026, doi: 10.48550/arXiv.2607.02748.
- [34] D. Kim, J.-Y. Choi, and J. Lee, “Trust Boundary Semantic Gaps: A Multi-dimensional Analysis and Mitigation for Security-by-Design,” arXiv:2607.01711, 2026, doi: 10.48550/arXiv.2607.01711.